

Fiche de révision

I Entrées / sorties

A Fonctions de base

On ouvre un fichier en mode lecture ou écriture avec `fopen` et on le referme avec `fclose`.

Fonction	Type	Exemple de code	Description de l'action
<code>putc</code>	Caractère	<code>putc('A', fp);</code>	Écrit un seul caractère ('A') dans le fichier <code>fp</code> .
<code>getc</code>	Caractère	<code>int c = getc(fp);</code>	Lit le caractère suivant depuis le fichier.
<code>fwrite</code>	Binaire	<code>fwrite(tab, sizeof(int), 10, fp);</code>	Écrit un bloc de données (ex: 10 entiers) vers le fichier.
<code>fread</code>	Binaire	<code>fread(tab, sizeof(int), 10, fp);</code>	Lit un bloc de données brutes pour remplir un tableau.

Table 1: Manipulation de caractères et de blocs binaires en C

✖ **Attention** ✖ En cas d'erreur, les deux premières fonctions renvoient EOF (équivalent à -1). C'est pour cela que `getc` retourne un `int` et pas un `char`.

📌 **Remarque** : `fwrite` et `fread` renvoient, sauf erreur, le nombre d'éléments effectivement écrits ou lus (ex: 10 pour le code ci-dessus).

🔧 **Application** : Pour récupérer le premier caractère d'un fichier, on peut faire :

```

1 int c;
2 c = getc(input);
3 char corresponding_char = (char) c; // Convertit l'entier en caractère
4 printf("Le premier caractère est : %c\n", corresponding_char);

```

📌 **Remarque** : Toutes ces fonctions n'écrivent pas directement sur le disque, elles utilisent un tampon en mémoire. Il faut appeler `fflush` pour forcer l'écriture immédiate.

B Formatage de données

Fonction	Entrée	Exemple de code	Description de l'action
<code>fprintf</code>	Fichier	<code>fprintf(fp, "Score: %d \n", score);</code>	Enregistre la valeur du score dans le fichier <code>fp</code> .
<code>fscanf</code>	Fichier	<code>fscanf(fp, "%d/%d/%d", &j, &m, &a);</code>	Récupère trois entiers séparés par des '/' depuis le fichier.
<code>snprintf</code>	Chaîne	<code>snprintf(buf, sizeof(buf), "%s", msg);</code>	Copie le message dans <code>buf</code> sans risquer de déborder.
<code>sscanf</code>	Chaîne	<code>sscanf(str, "%f %f", &x, &y);</code>	Convertit le texte de <code>str</code> en deux nombres flottants.

Table 2: Récapitulatif des fonctions de formatage en C et actions associées

📌 **Remarque** : `printf` et `scanf` sont des cas particuliers de `fprintf` et `fscanf` qui utilisent respectivement `stdout` et `stdin`.

II Les Unions

Une `union` est un type de donnée qui permet de stocker différents types de variables à la **même adresse mémoire**. Contrairement à une `struct`, une seule de ses variables peut être utilisée à la fois.

C'est-à-dire que c'est une structure où un seul des champs est réellement utilisé par une "instance" de l'union. L'union occupe en mémoire la taille du plus grand de ses champs, et tous les champs partagent le même espace mémoire.

Concept	Définition / Syntaxe	Conséquence en mémoire
Déclaration	<code>union u { int i; char *s; };</code>	<code>i</code> et <code>s</code> partagent exactement le même espace.
Taille	<code>sizeof(union u)</code>	Égale à la taille de son plus grand membre.
Accès	<code>mon_union.i = 10;</code>	Écrase toute valeur précédemment stockée dans <code>s</code> .

Table 3: Fonctionnement des unions en C

Remarque : On utilise souvent une union à l'intérieur d'une structure avec un "tag" (énumération ou entier) pour savoir quel membre de l'union est actuellement valide.

Application : Exemple d'une expression pouvant être soit un entier, soit une variable :

```

1 struct exp {
2     int type; // 0 pour constante, 1 pour variable
3     union {
4         int value;
5         char *name;
6     } data;
7 };
8
9 void print_exp(struct exp e) {
10     if (e.type == 0) printf("%d", e.data.value);
11     else printf("%s", e.data.name);
12 }

```

Attention Lire un membre d'une union qui n'est pas celui qui a été écrit en dernier provoque un comportement indéfini (interprétation erronée des octets).

III Expressions constantes

A Énumérations

Une `enum` est un type de donnée qui permet de définir un ensemble de constantes entières symboliques.

```

1 enum Color { RED, GREEN, BLUE }; // RED=0, GREEN=1, BLUE=2
2 enum Status { OK = 1, ERROR = -1, UNKNOWN = 0 }; // OK=1, ERROR=-1, UNKNOWN=0

```

B Constantes

On utilise le mot-clé `const` pour déclarer une variable dont la valeur ne peut pas être modifiée après son initialisation.

```

1 const int MAX_SIZE = 100; // MAX_SIZE est une constante entière
2 const char *GREETING = "Hello, World!"; // GREETING est une constante (string)

```

Remarque : On peut en réalité tout de même modifier la valeur pointée par un pointeur constant (ex: `GREETING[0] = 'h'`; est techniquement possible mais conduit à un comportement indéfini).

C Switch

Le switch permet d'éviter de longues séries de if-else lorsque l'on compare une expression à plusieurs constantes **entières** ou des valeurs d'**enum**.

```

1 switch (expression) {
2     case constant1:
3         // code à exécuter si expression == constant1 (int ou valeur d'enum)
4         break;
5     case 42:
6         // ...
7         break;
8     case 1 + 1: // équivalent à case 2:
9         // ...
10    case 'A': // équivalent à case 65:
11        // ...
12        break;
13    // ...
14    default:
15        // code à exécuter si expression ne correspond à aucun cas
16 }
```

IV Matrices (tableaux à 2 dimensions)

A Version "Java"

On peut faire un tableau à 2 dimensions en C de manière similaire à Java, c'est-à-dire en utilisant un tableau de pointeurs.

```

1 int **matrix = malloc(rows * sizeof(int *)); // Allocation du tableau de pointeurs
2 for (int i = 0; i < rows; i++) {
3     matrix[i] = malloc(cols * sizeof(int)); // Allocation de chaque ligne
4 }
```

❗ Remarque : Cette approche permet d'avoir des lignes de tailles différentes (tableau "jagged"), mais nécessite une gestion manuelle de la mémoire (libération avec free).

✗ Attention ✗ Cette méthode est plus lente que l'utilisation d'un tableau contigu en mémoire à cause de la fragmentation et du nombre d'allocations.

B Représentation contiguë en mémoire

Une autre approche pour représenter une matrice est d'utiliser un tableau à une dimension et de calculer les indices manuellement.

```

1 int *matrix = malloc(rows * cols * sizeof(int)); // Allocation d'un tableau contigu
2 // Accès à l'élément (i, j)
3 int value = matrix[i * cols + j];
```

❗ Remarque : Cette méthode est plus rapide et plus efficace en termes de mémoire, mais nécessite de calculer les indices à chaque accès.

✗ Attention ✗ Le langage C permet d'ailleurs d'utiliser cette méthode nativement via, par exemple, `int matrix[rows][cols];` qui est équivalent à un tableau contigu.